Data Structures - Trees: Revision Notes

A binary search tree (BST) is a binary tree in which every node has at most two children. For each node, all keys in the left subtree are less than the node’s key, and all keys in the right subtree are greater. This ordering property makes search, insertion, and deletion efficient.

Search in a BST: start at the root and compare the target key with the current node. If equal, the search succeeds. If the target is smaller, go left; if larger, go right. The average time complexity is O(log n) when the tree is balanced, but degrades to O(n) for a skewed tree.

Insertion follows the same path as search until reaching an empty spot, where the new node is attached as a leaf.

Balanced Trees and Traversals

An AVL tree is a self-balancing binary search tree where the heights of the two child subtrees of any node differ by at most one. After insertion or deletion, rotations restore the balance, guaranteeing O(log n) height.

Tree traversals visit nodes in a defined order. In-order traversal of a BST visits nodes in ascending key order: left subtree, node, right subtree. Pre-order visits the node before its subtrees; post-order visits the node after its subtrees. Level-order traversal uses a queue to visit nodes breadth-first, one level at a time.